

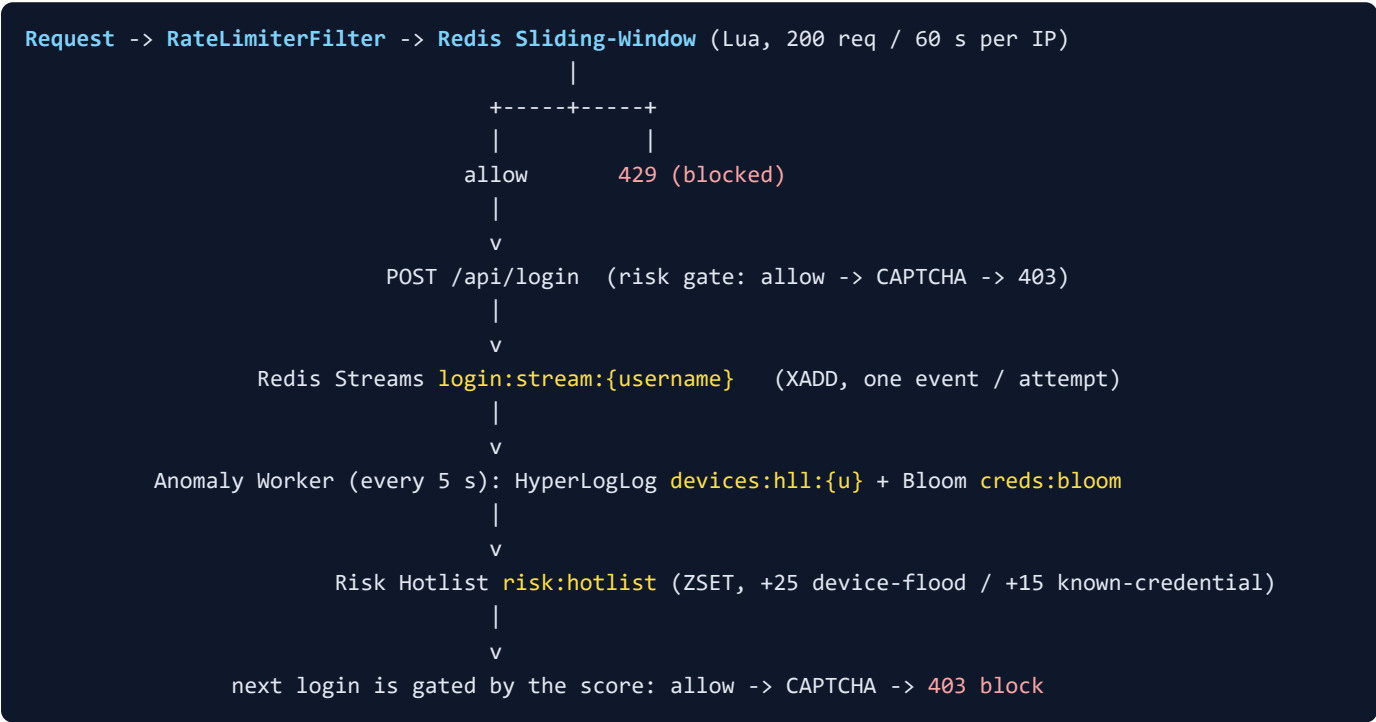
CacheGuard

Usage & Demonstration Guide

Redis-powered API gateway demo · Spring Boot 3.5 + Redis Stack · September 2026

CacheGuard protects login endpoints from request floods and credential-stuffing attacks. It combines an atomic per-IP **sliding-window rate limiter** with an **anomaly-detection pipeline** that scores suspicious accounts into a global risk hotlist and gates future logins according to that score — all backed by Redis and visualised by a live dashboard.

1. Architecture at a glance



2. Prerequisites

Requirement	Notes
Java 25	Runtime and build target (<code>java -version</code>)
Maven 3.6.3+	Builds and runs the app (<code>mvn -version</code>)
Docker + Compose	Provides Redis Stack (RedisBloom, streams, HyperLogLog)
Python 3.9+ and <code>requests</code>	Only for the demo script: <code>python -m pip install requests</code>

3. Quick start (2 minutes)

1 Start Redis Stack:

`docker compose up -d redis`

2 Start the backend:

```
mvn spring-boot:run
```

(or run everything in containers with `docker compose up --build`)

3 Verify it is up:

```
curl http://localhost:8080/api/health
OK
```

4 Open the frontend dashboard in a browser:

```
http://localhost:8080/dashboard.html
```

5 Run the demos (section 6) and watch the dashboard react live.

Frontend + backend share one address: `http://localhost:8080` — the dashboard is a static page served by the same Spring Boot process, so its live 3-second polling of `/api/stats` and `/api/hotlist` needs no CORS or extra server.

4. Using the dashboard

The dashboard auto-refreshes every 3 seconds and shows three panels:

Panel	What it tells you
Stats	Total requests seen by the gateway, requests blocked with <code>429</code> (plus <code>403</code> risk locks), flagged (CAPTCHA) logins, and the current hotlist size.
Top risk scores	Bar chart of the riskiest accounts from <code>GET /api/hotlist</code> .
Hotlist table	Ranked usernames and scores — run <code>--stuffing</code> and watch the target account appear here.

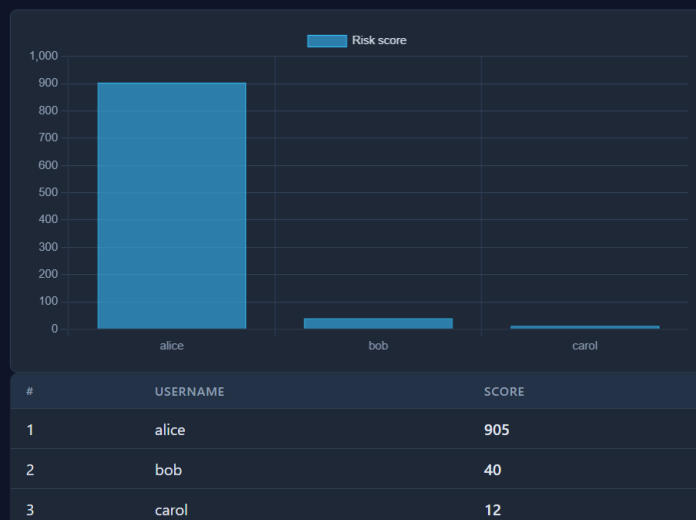
CacheGuard Dashboard

last updated 8:17:35 PM

STATS

METRIC	VALUE
Total requests	696
Blocked requests	417
Flagged logins	0
Hotlist size	3

TOP RISK SCORES



Live dashboard at <http://localhost:8080/dashboard.html> (captured from the running app)

5. Using the REST API

Endpoint	Purpose
GET /api/health	Redis PING health check → OK
POST /api/login	Login attempt: risk gate → credential check → event recording
GET /api/stats	Total / blocked / flagged counters + hotlist size
GET /api/hotlist?top=N	Top-N riskiest usernames (default 10)
GET /api/risk/{username}	Current risk score of one user
GET /api/events/{username}	Recent login events (ip, deviceId, result, timestamp)
GET /dashboard.html	The live dashboard UI

5.1 Health check

```
curl http://localhost:8080/api/health
OK
```

5.2 Login with the demo credentials

Seeded accounts: admin / secret , alice / password123 , bob / letmein .

```
curl -X POST http://localhost:8080/api/login \
-H "Content-Type: application/json" \
-d '{"username":"alice","password":"password123","ip":"198.51.100.10","deviceId":"laptop-01"}'

{"success":true,"username":"alice","message":"Authentication successful",
"timestamp":"2026-09-25T14:35:10Z","requiresCaptcha":false}
```

A wrong password still answers `200` — the attempt is **recorded** into the login stream, HyperLogLog and Bloom filter so the anomaly pipeline can learn from it:

```
... "password":"wrong-guess" ...
{"success":false,"username":"alice","message":"Invalid credentials", ... ,"requiresCaptcha":false}
```

5.3 Risk gate responses (once an account is scored)

```
Score >= 30 → 200 {"message":"CAPTCHA verification required","requiresCaptcha":true}
Score >= 60 → 403 {"message":"Account locked due to suspicious activity","riskScore":905.0}
```

5.4 Inspect risk and events

```
curl http://localhost:8080/api/risk/alice
{"username":"alice","score":905.0}

curl "http://localhost:8080/api/hotlist?top=3"
{"count":3,"entries":[{"username":"alice","score":905.0},
{"username":"bob","score":40.0},{username":"carol","score":12.0}]}

curl http://localhost:8080/api/stats
{"totalRequests":696,"blockedCount":417,"flaggedCount":0,"hotlistSize":3}
```

6. Running the demonstration script

`scripts/load_test.py` drives the running app with synthetic traffic so you can prove both defence layers end to end. Install the dependency once, then run each mode independently (exactly one mode flag per run):

```
python -m pip install requests
```

MODE 1 Normal traffic — `--normal`

A few requests/second from five rotating IPs. Nothing should ever be rate-limited.

```
python scripts/load_test.py --normal

[normal] sending 20 requests to /api/health
[normal] rotating 5 source IPs, delay=0.3s
[normal] done -> 200: 20
[normal] OK: all responses were 200, no rate limiting triggered
```

MODE 2 Rate-limit attack — `--attack`

Hundreds of requests/second from **one fixed IP** (203.0.113.10). The gateway allows 200 requests per 60-second window per IP+endpoint, so the burst splits into `200` vs `429` responses:

```
python scripts/load_test.py --attack

[attack] sending 600 requests to /api/health from fixed IP 203.0.113.10
[attack] 8 workers, delay=0.002s per request
[attack] finished 600 requests in 2.3s (~265 req/s)
[attack] 200 OK: 200
[attack] 429 rate-limited: 400
[attack] OK: rate limiting is working: 400 of 600 requests were blocked with 429
```

Re-running within 60 seconds starts against an already-exhausted window (expect all 429 s). Wait a minute or use a fresh IP: `--ip 203.0.113.11`. Watch **Blocked requests** climb on the dashboard while this runs.

MODE 3 Credential stuffing — `--stuffing`

One username (`alice`) and one guessed password sprayed from **many unique fake deviceIds** with rotating IPs, so the per-IP limiter never fires — detection has to come from the anomaly pipeline. The script finishes by querying the hotlist itself:

```
python scripts/load_test.py --stuffing

[stuffing] sending 40 login attempts for user 'alice'
[stuffing] one password, 40 unique fake deviceIds, rotating fake IPs, delay=0.05s
[stuffing] done -> 200: 23, 403: 17
[stuffing] the risk gate kicked in mid-attack (CAPTCHA/lock) while the risk score was climbing
[stuffing] waiting 7s for the anomaly worker (it polls every 5s)...
[stuffing] GET /api/hotlist?top=10 -> 1 entries (top flagged usernames)
#1 alice score=905.0 <-- stuffed account
[stuffing] OK: 'alice' is on the hotlist - anomaly detection flagged the stuffing attempt
```

Exit code `0` only when the targeted username appears on the hotlist. The app also logs each detection, e.g. `[StreamWorker] user=alice device spike detected (40 > 3), +25 risk pts`.

Useful options

Option	Default	Meaning
<code>--url</code>	<code>http://localhost:8080</code>	Base URL of the app
<code>--count</code> / <code>--delay</code>	mode-specific	Number of requests / delay between them
<code>--ip</code> / <code>--workers</code>	<code>203.0.113.10</code> / <code>8</code>	Attacker IP and threads for <code>--attack</code>
<code>--username</code> / <code>--password</code>	<code>alice</code> / <code>Spring2026!</code>	Target account and guessed password for <code>--stuffing</code>
<code>--wait</code> / <code>--top</code>	<code>7</code> / <code>10</code>	Seconds to wait for the worker / hotlist entries fetched

7. How risk scoring works

Signal	Points	Detected by
Device flood — more than 3 distinct devices for one account (per scored event)	+25	HyperLogLog <code>devices:h1l:{u}</code> (PFCOUNT) in the worker
Known credential — the same username:password pair seen before	+15	Bloom filter <code>creds:bloom:attempts</code> verdict recorded at write time

Score range	Login behaviour	HTTP
0 – 29	Credentials checked normally	200 / 401-style body
30 – 59	CAPTCHA challenge required, credentials not checked	200 <code>requiresCaptcha:true</code>
60 and above	Account locked	403

Thresholds are configured in `src/main/resources/application.yml` under `cacheguard.risk` (`deviceCountThreshold: 3`, `allowScoreMax: 30.0`, `captchaScoreMax: 60.0`). The rate limiter allows **200 requests per 60 seconds per IP+endpoint** (also visible as `ratelimit:{ip}:{endpoint}` in Redis).

8. Reset & troubleshooting

```
docker exec cacheguard-redis redis-cli FLUSHALL # wipe all demo state
```

- **Results look "sticky" between runs** — Redis keeps risk scores, rate-limit windows and device counts; flush to start fresh. The rate-limit window also expires by itself after 60 s without traffic.
- **No 429 s in `--attack`** — the limit is 200/window; raise `--count` above 200.
- **Hotlist empty after `--stuffing`** — the worker polls every 5 s; increase `--wait`.
- **Connection refused** — the app or Redis is not up yet (`docker compose ps`, app logs).
- **Run the test suite** — `mvn test` (uses Testcontainers when Docker is available, falls back to localhost:6379, skips when no Redis is reachable).